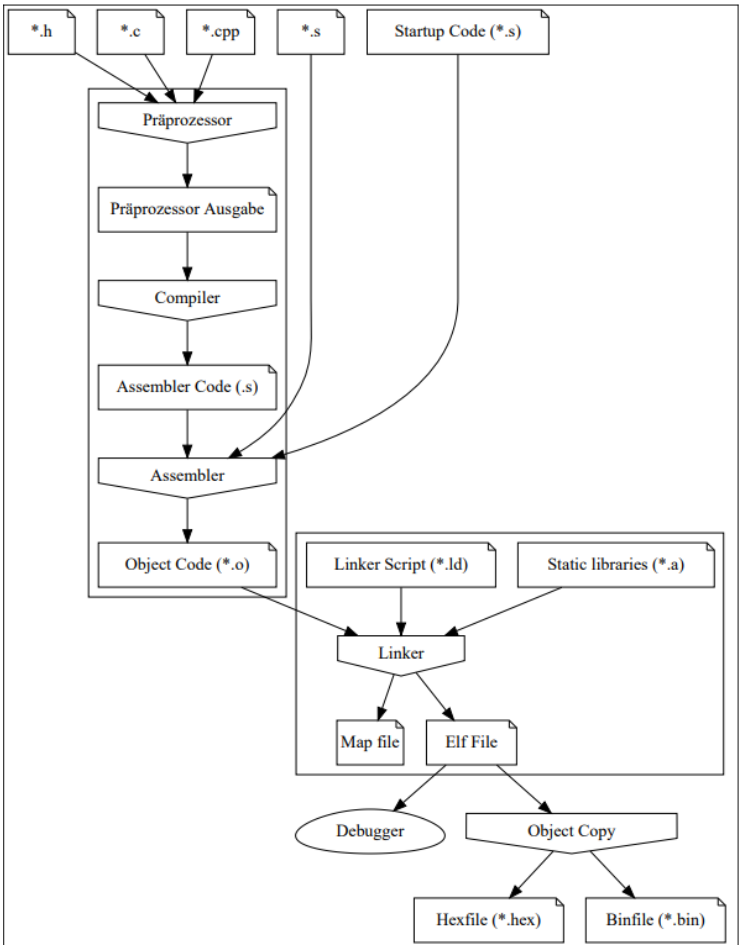
Informatik 3 Zusammenfassung

1. Entstehung eines ausführbaren Programms



2. Referenzen

- interne Zeiger auf Variablen
- Im Gegensatz zu Zeigern kann eine Referenz später im Programm nicht auf ein anderes Objekt referenzieren
- Referenzen können nie NULL werden, sie zeigen immer auf gültige Objekte.
- Sie müssen beim Erstellen initialisiert werden
- Ist ein Alias für die erstellte Variable, quasi ein Abbild
- Ändert Wert immer mit der zugehörigen Variable

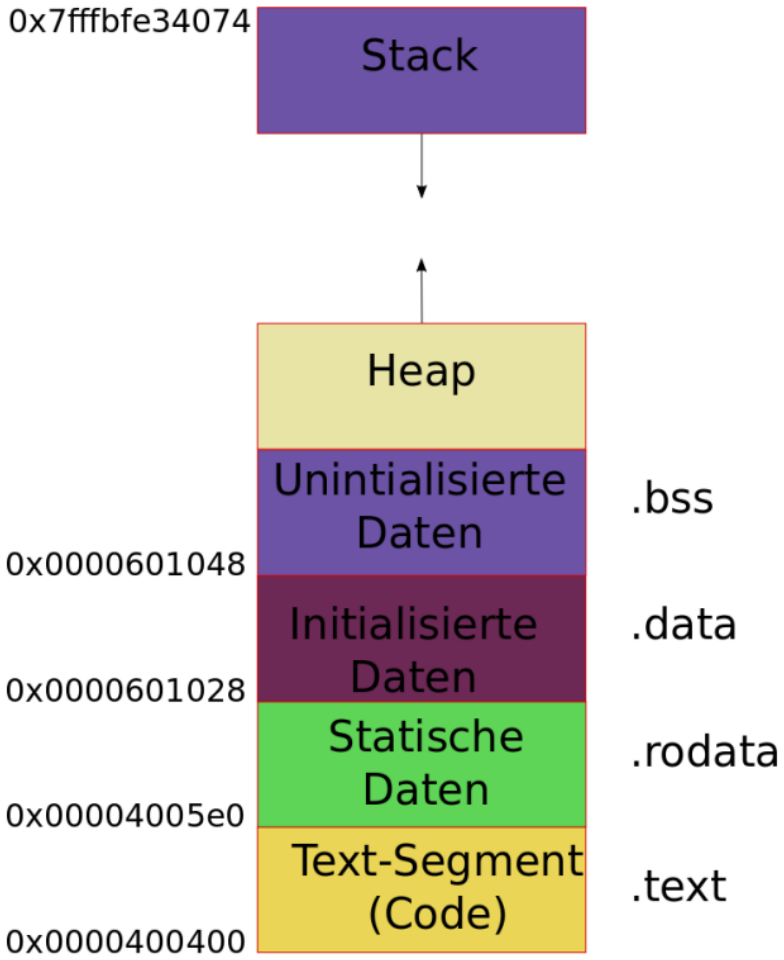
3. CMake

- Generator für verschiedene Buildsysteme
- Unabhängig von der genutzten DIE
- Unterstützt somit Crossplattform-Arbeit an einem Projekt, z.B. mit VS Code und XCode(MacOS)

4. Speicherverwaltung

- Vier Bereiche:
    - Programmspeicher
    - Globaler Speicher (für globale Variablen)
    - Heap (dynamische Speicherverwaltung)
    - Stack/Stapelspeicher
- Beim Ausführen eines Programmes wird dieses in den Arbeitsspeicher geladen, in Segmente aufgeteilt und von dort ausgeführt
- Segmente/Sektionen für Daten:
    - .text
    - .data
    - .bss
    - .comment
    - .note.GNU-stack
    - .eh-frame
- .text:
    - enthält Funktionen
    - enthält Programmcode
- .data:
    - enthält alle globalen und statischen Variablen mit vordefiniertem Wert
    - somit alle Variablen, die außerhalb einer Funktion (auch main) definiert wurden
- .bss:
    - enthält alle nicht oder auf NULL initialisierten Variablen
    - alle nicht initialisierten Variablen werden beim Ausführen automatisch auf NULL initialisiert
- **alle const Variablen** landen in **.rodata**
- alle Variablen, die innerhalb von {} (d.h. innerhalb von Funktionen) deklariert werden, werden im Stack Speicher abgelegt (am Ende des Blocks verlierend diese ihre Gültigkeit) -> statische Speicherverwaltung
- *return* von Zeigern auf lokale Variablen der Funktion ist deshalb **verboten**, da die Variable nach Beenden der Funktion ja nicht mehr gültig ist
- Aufrufparameter einer Funktion werden mit in den Stack übergeben (Variablen, Rücksprungadresse)
- Sollten die Aufrufparameter große Objekte sein, wird ein Zeiger auf sie übergeben, anstatt sie in den Stack zu kopieren
- Stackgröße: 1MB in VS, 8MB in Linux

- Stack execution protection: Verhindert Ausführung von Programmen im Stack, schützt vor Stack-Overflow Angriffen
- **HEAP:** Dynamischer Speicher, welcher während der Laufzeit allokiert werden kann
    - Wird in C mit *malloc* allokiert und mit *free* freigegeben
    - In C++ mit *new* allokiert und mit *delete* freigegeben
    - **Niemals C und C++ Befehle mischen!**
    - Mit *new* allozierter Speicher muss mit *delete* freigegeben werden, sonst entsteht ein sog. *Memory-Leak*
    - Nach dem *delete* sollte der Pointer auf NULL initialisiert werden, da ein erneuter Zugriff zu einem Laufzeitfehler führt
    - Falls *new* nicht genug Speicher allokieren kann, wird es versuchen mehrere Speicherteile zu finden, die zusammen reichen, oder *void *new_handler* wird versuchen, Speicher freizuräumen. Falls das nicht gelingt, wird das Programm sicher beendet (*new_handler* muss dafür initialisiert sein, falls nicht wirft *new* eine Exception)
    - Wird *new* mit dem Parameter *std::no_throw* aufgerufen, wird keine Exception geworfen, sondern ein NULL-Pointer returned



- HEAP und Stack wachsen sich entgegen, da beide nicht unendlich groß werden können

5. Objektorientierung

- **Klasse**: objektorientierte Programmierung eines abstrakten Modells bzw. eines Bausplans für Objekte mit Attributen und Methoden
    - **Konstruktoren:** Funktion zum Instanzieren eines Objektes
    - **Destruktor:** löst Objekt auf, sobald es nicht mehr verwendet wird
- **Objekt:** Exemplar eines Datentyps/Klasse, werden während der Laufzeit erzeugt
- **Operatoren:**
    - Bereichsoperator: Namespace-Bezeichnung vor Datentypen, mit :: getrennt
    - Überladen: Operatoren wie „=“ können durch eigene Funktionen überladenw erden
- Die großen Drei: Kopierkonstruktor, Destruktor, Zuweisungsoperator
    - Die Implementierung einer dieser drei Methoden bedeutet, dass mit dynamischem Speicher gearbeitet wird, was die anderen beiden zwingend notwendig macht

6. Vererbung

- Die vererbende Klasse ist die sog. Basisklasse
- Bei public-Vererbung handelt es sich immer um eine **ist-eine** Beziehung
- Die abgeleitete Klasse kann nur auf Member der Basisklasse zugreifen, die als *public* oder *protected* deklariert sind
- *Virtual* ist das Schlüsselwort, was einer abgeleiteten Klasse das Überschreiben einer Methode der Basisklasse ermöglicht
- *Override* wird an die Deklaration der abgeleiteten Mehtode angehängt, es symbolisiert, dass die Basismethode überschrieben werden soll
- Wird mind. Eine Methode einer mit *virtual* beschrieben und mit 0 initialisiert, handelt es sich um eine rein virtuelle Methode und somit um eine abstrakte Klasse, von welcher keine Objekte entstehen können
- Sind **alle Methoden** pure virtual und *public* handelt es sich um ein *Interface*
- Interfaces sind „Verträge“, nach welchen Klassen gebaut werden, diese müssen dann alle pure virtual Funktionen des Interface implementieren

7. Templates

- Vorlagen für Funktionen/Mehtoden/Klassen ohne vorgegebenn Datentyp

8. Container

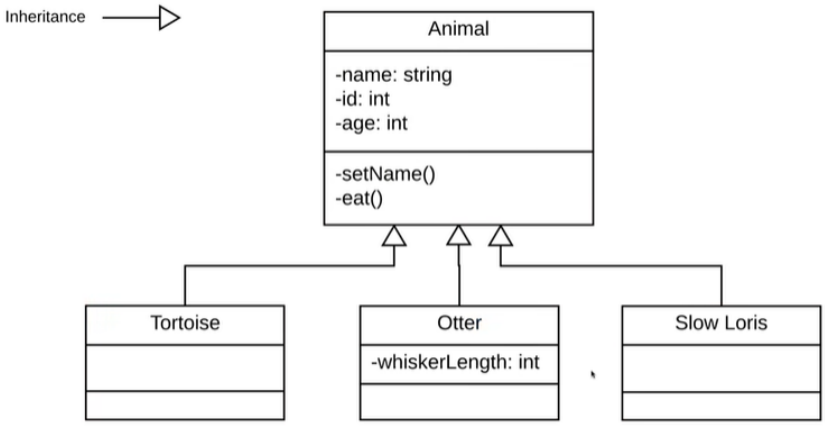
- Ein Container ist ein flexibles Array (Bsp: std::vector)

9. UML und Strukturdiagramme

- Klassendiagramm:
    - Zeigt alle benötigten Klassen, ihr Verhältnis und die Verbindungen
    - Die Klasse wird als rechteck dargestellt, oben Name, darunter Attrbiute, unten Methoden:

- Vor die Attribute und Methoden werden je nach Sichtbarkeit Vorzeichen gesetzt:

- Die Attributen Namen werden immer klein geschrieben, danach wird nach dem Doppelpunkt der Datentyp genannt
- **Vererbung:** Vererbung wird in UML mit Pfeilen dargestellt, die member der Basisklasse müssen nicht übernommen werden:



- **Assoziation:** Verbindung/Beziehung zweier Klassen (z.B. Otter ->isst->Slow Loris), dargestellt durch Linie
- **Aggregation:** spezielle Assoziation, beinhaltet z.B. eine Gruppe von anderen Klassenobjekten (Bsp: Otter-Herde -> beinhaltet Otter, diese können auch ohne Herde existieren)
- **Komposition:** Eine Assoziation, bei der das Objekt ohne das andere **nicht existieren kann** (Bsp: Badezimmer kann ohne Haus nicht existieren: Badezimmer ◆ Haus)
- **Multiplizität:**
    - Beschränkt die Anzahl an Child-Objekten pro Parent-Objekt bei Kompositionen (Bsp: Ein Haus kann nur 1 Badezimmer haben), markiert mit der Anzahl auf der Linie.
    - Wenn ein Parent-Objekt eine Mindestanzahl an Child-Objekten haben soll (mind. 2 Badezimmer pro Haus), wird die Multiplizität mit „2..*“ gekennzeichnet
    - Multiplizität geht auch in einer Range, z.B. von 2-5 Badezimmern, dann ist es „2..5“